Operating Systems
Week 15 Lecture 15

Junaid Abdul Wahid

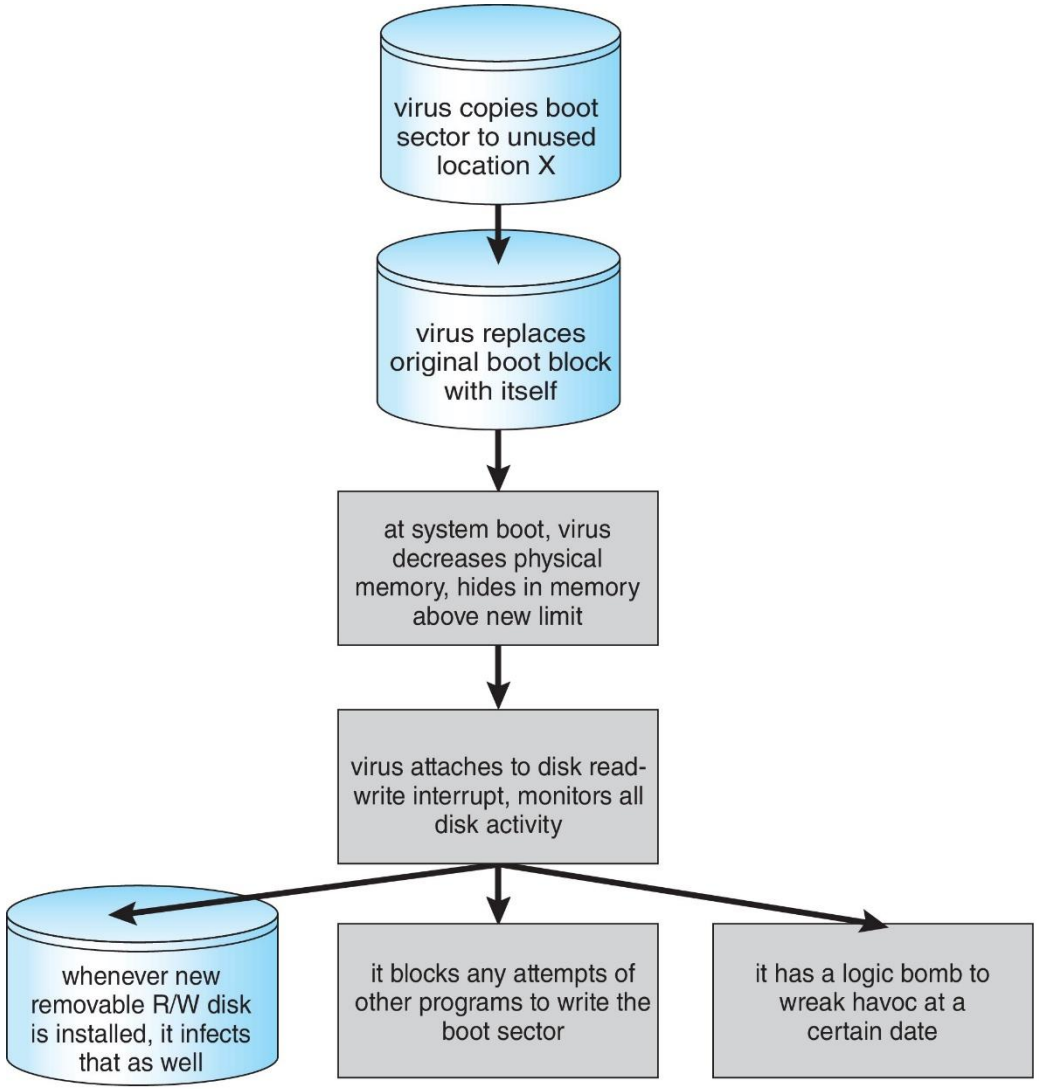
Previous

- Exploring the structure of an operating system's I/O subsystem

Today Agenda

- Explore the structure of an operating system's I/O subsystem

A Boot-sector Computer Virus



The Threat Continues

Attacks still common, still occurring

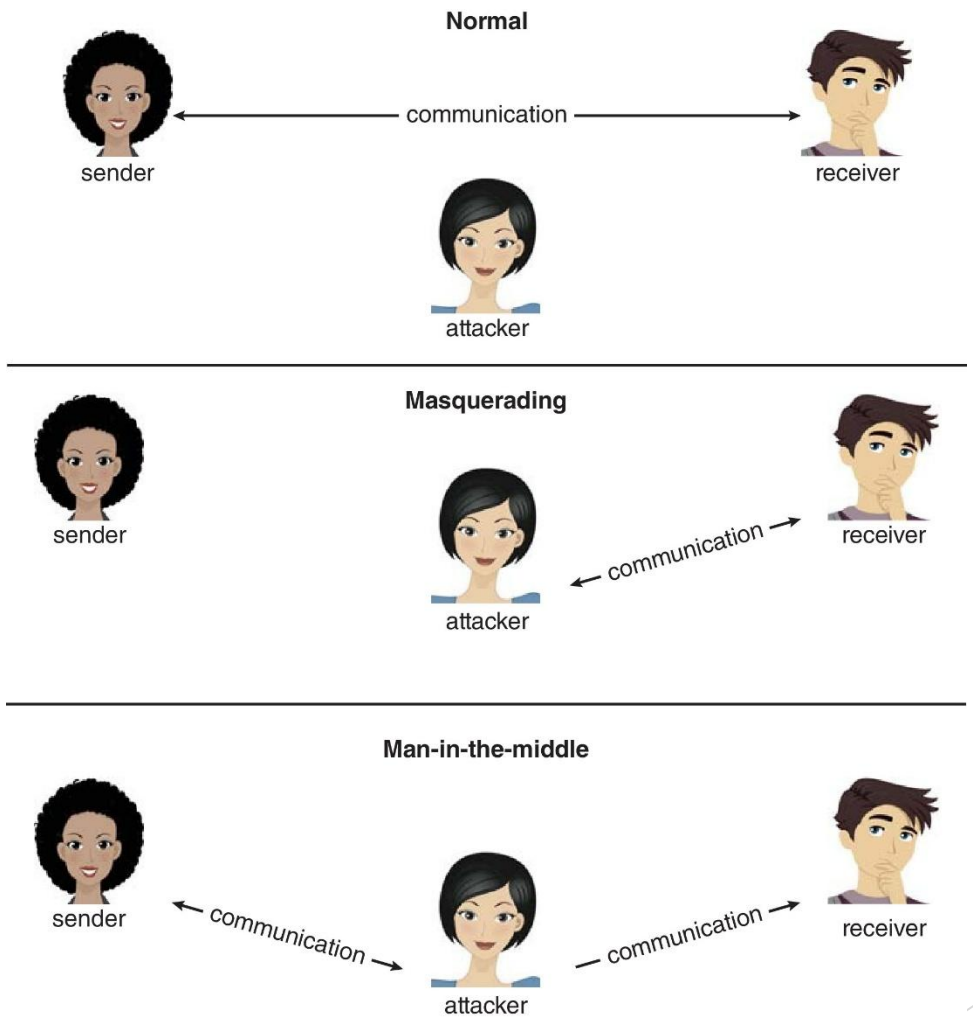
Attacks moved over time from science experiments to tools of organized crime

- Targeting specific companies
- Creating botnets to use as tool for spam and DDOS delivery
- **Keystroke logger** to grab passwords, credit card numbers

Why is Windows the target for most attacks?

- Most common
- Everyone is an administrator
  - Licensing required?
- **Monoculture** considered harmful

Standard Security Attacks



Cryptography as a Security Tool

- Broadest security tool available
  - Internal to a given computer, source and destination of messages can be known and protected
    - OS creates, manages, protects process IDs, communication ports
  - Source and destination of messages on network cannot be trusted without cryptography
    - Local network – IP address?
      - Consider unauthorized host added
    - WAN / Internet – how to establish authenticity
      - Not via IP address

Cryptography

Means to constrain potential senders (*sources*) and / or receivers (*destinations*) of *messages*

- Based on secrets (**keys**)
- Enables
  - Confirmation of source
  - Receipt only by certain destination
  - Trust relationship between sender and receiver

Authentication

- Constraining set of potential senders of a message
  - Complementary to encryption
  - Also can prove message unmodified
- Algorithm components
  - A set K of keys
  - A set M of messages
  - A set A of authenticators
  - A function $S : K \rightarrow (M \rightarrow A)$
    - That is, for each $k \in K$, S_k is a function for generating authenticators from messages
    - Both S and S_k for any k should be efficiently computable functions
  - A function $V : K \rightarrow (M \times A \rightarrow \{\text{true, false}\})$. That is, for each $k \in K$, V_k is a function for verifying authenticators on messages
    - Both V and V_k for any k should be efficiently computable functions

Authentication (Cont.)

For a message *m*, a computer can generate an authenticator $a \in A$ such that $V_k(m, a)$ = `true` only if it possesses *k*

Thus, computer holding *k* can generate authenticators on messages so that any other computer possessing *k* can verify them

Computer not holding *k* cannot generate authenticators on messages that can be verified using V_k

Since authenticators are generally exposed (for example, they are sent on the network with the messages themselves), it must not be feasible to derive *k* from the authenticators

Practically, if $V_k(m,a)$ = **`true`** then we know *m* has not been modified and that send of message has *k*

If we share *k* with only one entity, know where the message originated

Authentication – Hash Functions

Basis of authentication

Creates small, fixed-size block of data **message digest** (**hash value**) from *m*

Hash Function *H* must be collision resistant on *m*

- Must be infeasible to find an $m' \neq m$ such that $H(m) = H(m')$

If $H(m) = H(m')$, then $m = m'$

- The message has not been modified

Common message-digest functions include **MD5**, which produces a 128-bit hash, and **SHA-1**, which outputs a 160-bit hash

Not useful as authenticators

- For example *H(m)* can be sent with a message
  - But if *H* is known someone could modify *m* to *m'* and recompute *H(m')* and modification not detected
  - So must authenticate *H(m)*

Authentication - MAC

Symmetric encryption used in **message-authentication code** (**MAC**) authentication algorithm

Cryptographic checksum generated from message using secret key

- Can securely authenticate short values

If used to authenticate *H(m)* for an *H* that is collision resistant, then obtain a way to securely authenticate long message by hashing them first

Note that *k* is needed to compute both S_k and V_k, so anyone able to compute one can compute the other

Authentication – Digital Signature

Based on asymmetric keys and digital signature algorithm

Authenticators produced are **digital signatures**

Very useful – ***anyone*** can verify authenticity of a message

In a digital-signature algorithm, computationally infeasible to derive k_s from k_v

- V is a one-way function
- Thus, k_v is the public key and k_s is the private key

Consider the RSA digital-signature algorithm

- Similar to the RSA encryption algorithm, but the key use is reversed
- Digital signature of message $S_{ks}(m) = H(m)^{k_s} \bmod N$
- The key k_s again is a pair (d, N), where N is the product of two large, randomly chosen prime numbers p and q
- Verification algorithm is $V_{kv}(m, a)$ $(a^{k_v} \bmod N = H(m))$
  - Where k_v satisfies $k_v k_s \bmod (p-1)(q-1) = 1$

$\stackrel{?}{=}$

Authentication (Cont.)

Why authentication if a subset of encryption?

- Fewer computations (except for RSA digital signatures)
- Authenticator usually shorter than message
- Sometimes want authentication but not confidentiality
  - Signed patches et al
- Can be basis for **non-repudiation**

Key Distribution

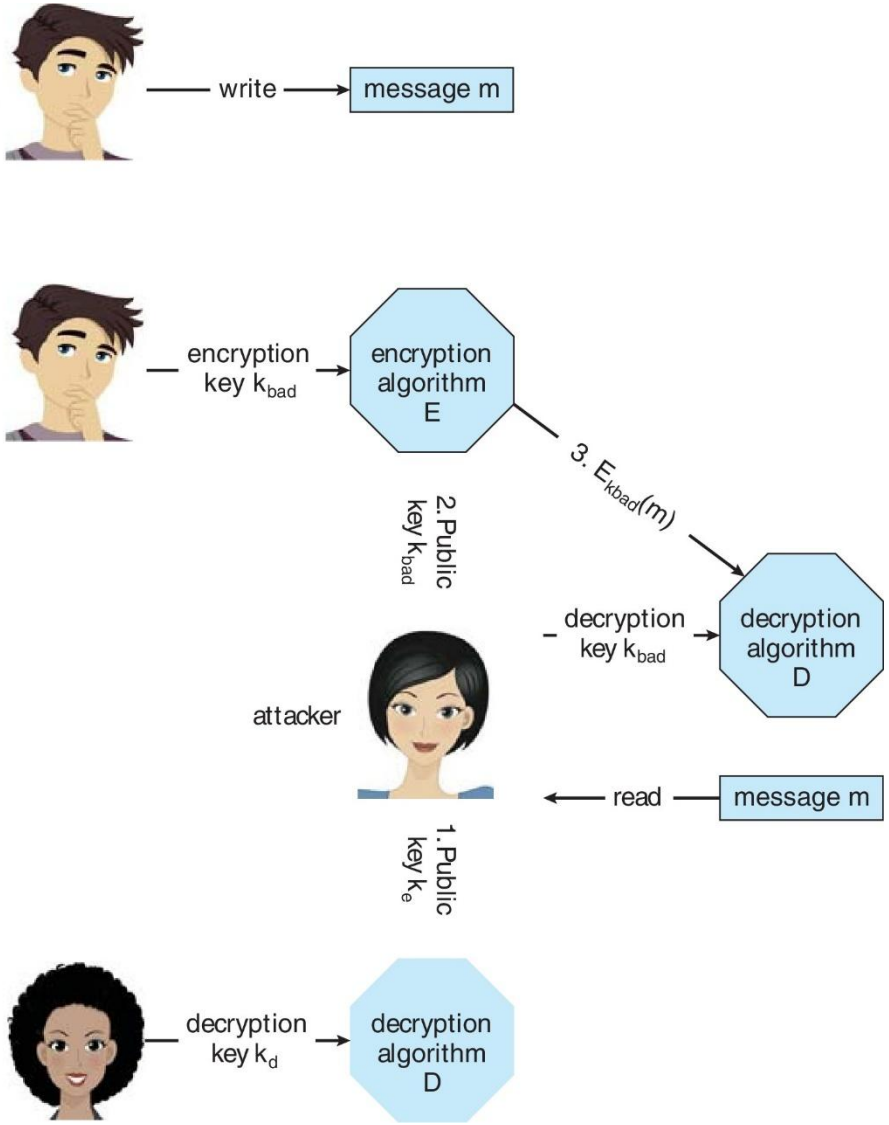
- Delivery of symmetric key is huge challenge
  - Sometimes done **out-of-band**
- Asymmetric keys can proliferate – stored on **key ring**
  - Even asymmetric key distribution needs care – man-in-the-middle attack

Digital Certificates

Proof of who or what owns a public key
Public key digitally signed a trusted party
Trusted party receives proof of identification from entity and certifies that public key belongs to entity
Certificate authority are trusted party – their public keys included with web browser distributions

- They vouch for other authorities via digitally signing their keys, and so on

Man-in-the-middle Attack on Asymmetric Cryptography



User Authentication

Crucial to identify user correctly, as protection systems depend on user ID

User identity most often established through **passwords**, can be considered a special case of either keys or capabilities

Passwords must be kept secret

- Frequent change of passwords
- History to avoid repeats
- Use of "non-guessable" passwords
- Log all invalid access attempts (but not the passwords themselves)
- Unauthorized transfer

Passwords may also either be encrypted or allowed to be used only once

- Does encrypting passwords solve the exposure problem?
  - Might solve **sniffing**
  - Consider **shoulder surfing**
  - Consider Trojan horse keystroke logger
  - How are passwords stored at authenticating site?

Passwords

- Encrypt to avoid having to keep secret
  - But keep secret anyway (i.e. Unix uses superuser-only readably file `/etc/shadow`)
  - Use algorithm easy to compute but difficult to invert
  - Only encrypted password stored, never decrypted
  - Add "salt" to avoid the same password being encrypted to the same value
- One-time passwords
  - Use a function based on a seed to compute a password, both user and computer
  - Hardware device / calculator / key fob to generate the password
    - Changes very frequently
- Biometrics
  - Some physical attribute (fingerprint, hand scan)
- Multi-factor authentication
  - Need two or more factors for authentication
    - i.e., USB "dongle", biometric measure, and password

Passwords (Cont.)

STRONG AND EASY TO REMEMBER PASSWORDS

It is extremely important to use strong (hard to guess and hard to shoulder surf) passwords on critical systems like bank accounts. It is also important to not use the same password on lots of systems, as one less important, easily hacked system could reveal the password you use on more important systems. A good technique is to generate your password by using the first letter of each word of an easily remembered phrase using both upper and lower characters with a number or punctuation mark thrown in for good measure. For example, the phrase "My girlfriend's name is Katherine" might yield the password "Mgn.isK!". The password is hard to crack but easy for the user to remember. A more secure system would allow more characters in its passwords. Indeed, a system might also allow passwords to include the space character, so that a user could create a **passphrase** which is easy to remember but difficult to break.

Implementing Security Defenses

Defense in depth is most common security theory – multiple layers of security

Security policy describes what is being secured

Vulnerability assessment compares real state of system / network compared to security policy

Intrusion detection endeavors to detect attempted or successful intrusions

- **Signature-based** detection spots known bad patterns
- **Anomaly detection** spots differences from normal behavior
  - Can detect **zero-day** attacks
- **False-positives** and **false-negatives** a problem